

AWS EC2 Deployment Runbook

Sai Kumar Studio — FastAPI Backend + Nginx (backend only, no frontend on the VM)

This is the exact, tested sequence used to stand up the backend on a fresh Ubuntu 24.04 EC2 VM: FastAPI running under systemd, Nginx reverse-proxying **/api**, **/admin** and **/media** to it. No frontend files are deployed to the VM — the frontend is hosted separately (local dev server or GitHub Pages) and points at this backend's URL.

Run the commands under “Local machine” from your own PC (Git Bash / WSL / any POSIX shell with `ssh+scp`). Run the commands under “On the VM” over SSH once connected.

0. Prerequisites

- An AWS account with permission to launch EC2 instances.
- A local copy of this project with a filled-in backend/ `.env` (this file is gitignored and is never cloned from GitHub — it must be copied to the VM directly).
- A terminal with `ssh`, `scp` and `tar` available (Git Bash on Windows works).

1. Launch the EC2 instance (AWS Console)

- EC2 → Launch Instance.
- AMI: **Ubuntu Server 24.04 LTS**, 64-bit (x86).
- Instance type: **t3.small** is a comfortable default (t3.micro works too, and is free-tier eligible).
- Key pair: create a new one (or reuse an existing one) and download the `.pem` file. AWS only lets you download it once.
- Network settings → Auto-assign public IP: **Enable**.
- Firewall (security group) — allow:

Type	Port	Source
SSH	22	Your IP (recommended) or 0.0.0.0/0 short-term
HTTP	80	0.0.0.0/0 (Anywhere)
HTTPS	443	0.0.0.0/0 (Anywhere)

- Storage: bump to **20 GiB gp3** (default 8 GiB is tight).
- Launch, then copy the instance's **public IPv4 address** — you'll need it below.

2. Set variables (local machine)

Set these once per terminal session; every later command reuses them.

```
EC2_IP="<new-vm-public-ip>"
PEM_PATH="/c/Users/mohan/Downloads/<your-key>.pem"
LOCAL_PROJECT_DIR="/d/Gen AI/Studio/studio-backed"
REMOTE_USER="ubuntu"
```

3. Test SSH, then install system packages

```
ssh -i "$PEM_PATH" -o StrictHostKeyChecking=accept-new "$REMOTE_USER@$EC2_IP" "echo connected"

ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" \
    "sudo apt-get update -y && sudo apt-get install -y python3-venv python3-pip nginx git"
```

4. Package and upload the backend + admin code

Only backend/ and admin/ are needed on this VM — the admin dashboard is served by FastAPI itself, and no customer-facing frontend is deployed here.

```
cd "$LOCAL_PROJECT_DIR"

tar -czf /tmp/studio-deploy.tar.gz \
    --exclude='backend/venv' --exclude='backend/.venv' \
    --exclude='__pycache__' --exclude='*.pyc' \
    backend admin

scp -i "$PEM_PATH" /tmp/studio-deploy.tar.gz "$REMOTE_USER@$EC2_IP:/home/ubuntu/"

ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" \
    "mkdir -p /home/ubuntu/app && \
    tar -xzf /home/ubuntu/studio-deploy.tar.gz -C /home/ubuntu/app && \
    rm /home/ubuntu/studio-deploy.tar.gz"
```

backend/.env is gitignored and NOT part of the repo history, but it IS included by this tar (it lives on disk in backend/). Double check it has the right values (DATABASE_URL, CORS_ORIGINS, Cloudfare/R2/Razorpay keys, etc.) before uploading — see Step 9.

5. Create the Python virtual environment

```
ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" \
    "cd /home/ubuntu/app/backend && \
    python3 -m venv venv && \
    ./venv/bin/pip install --upgrade pip -q && \
    ./venv/bin/pip install -r requirements.txt -q"
```

6. Allow Nginx to traverse the home directory

Ubuntu's default /home/ubuntu permissions (750) block the www-data user Nginx runs as. This single command fixes it:

```
ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" "chmod o+x /home/ubuntu"
```

7. Create the systemd service for the backend

On the VM, create `/etc/systemd/system/studio-backend.service` with this content (e.g. `sudo nano ...` and paste):

```
[Unit]
Description=Sai Kumar Studio FastAPI backend
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/app/backend
ExecStart=/home/ubuntu/app/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
Restart=always
RestartSec=3
EnvironmentFile=-/home/ubuntu/app/backend/.env

[Install]
WantedBy=multi-user.target
```

Do NOT add `--workers 2` (or more) to the `ExecStart` line. On a small instance this hangs forever at “Waiting for application startup” — a fork-safety issue with the DB connection pool being created before the worker processes fork. A single worker (the default, no flag) starts reliably, just slowly: allow 30-40 seconds on a cold start for the first DB connection to the Supabase pooler.

8. Create the Nginx site config

Create `/etc/nginx/sites-available/studio` with this content. It only proxies API/admin/media paths — everything else 404s, since no frontend lives here:

```

server {
    listen 80 default_server;
    server_name _;
    client_max_body_size 20M;

    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /admin/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /media/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
    }

    location / {
        return 404;
    }
}

```

9. Point CORS at your frontend origins

In backend/ .env on the VM, make sure CORS_ORIGINS lists every origin that will call this API (your local dev port, your GitHub Pages URL, a future production domain), comma-separated, e.g.:

```

CORS_ORIGINS=http://localhost:5500,http://127.0.0.1:5500,\
https://<your-username>.github.io

```

10. Enable everything and start

```

ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" "\
sudo systemctl daemon-reload && \
sudo systemctl enable --now studio-backend && \
sudo ln -sf /etc/nginx/sites-available/studio /etc/nginx/sites-enabled/studio && \
sudo rm -f /etc/nginx/sites-enabled/default && \
sudo nginx -t && \
sudo systemctl restart nginx"

```

11. Verify

```

curl http://$EC2_IP/api/health      # -> {"status":"ok"}
curl http://$EC2_IP/api/docs       # -> Swagger UI HTML
curl http://$EC2_IP/api/catalog/gifts # -> real catalog JSON

```

If health returns nothing for the first ~30-40 seconds after (re)starting the service, that's expected — wait and retry rather than assuming it's broken (see the note in Step 7).

12. Optional: HTTPS via Cloudflare Tunnel (no domain needed)

Skip this if the frontend calling the API is plain HTTP (e.g. a local dev server). Required if the frontend is served over HTTPS (GitHub Pages, most real domains) — browsers block HTTPS pages from calling a plain-HTTP API as “mixed content”.

```
ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" '\
CF_URL="https://github.com/cloudflare/cloudflared/releases"
CF_URL="$CF_URL/latest/download/cloudflared-linux-amd64.deb"
curl -sSL -o /tmp/cloudflared.deb "$CF_URL"
sudo dpkg -i /tmp/cloudflared.deb'
```

Create `/etc/systemd/system/cloudflared-studio.service`:

```
[Unit]
Description=Cloudflare Tunnel for Sai Kumar Studio backend
After=network.target studio-backend.service
Requires=studio-backend.service

[Service]
Type=simple
User=ubuntu
ExecStart=/usr/local/bin/cloudflared tunnel --url http://127.0.0.1:8000 \
--logfile /home/ubuntu/cloudflared.log --no-autoupdate
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
```

```
ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" "\
sudo systemctl daemon-reload && \
sudo systemctl enable --now cloudflared-studio && \
sleep 8 && grep trycloudflare /home/ubuntu/cloudflared.log"
```

This is a free “quick tunnel” — the `https://...trycloudflare.com` URL changes every time the cloudflared service restarts (crash, reboot). Fine for testing; for a permanent URL, buy a domain and use Let's Encrypt/certbot instead.

13. Point the frontend at this backend

Edit `js/shared/api-config.js` in the frontend repo:

```
window.SAI_API_BASE = "http://<EC2_IP>";
// or, if using the Cloudflare Tunnel / a real domain:
// window.SAI_API_BASE = "https://<tunnel-or-domain>";
```

Commit and push so any hosted frontend (e.g. GitHub Pages) picks it up.

14. Redeploying after future code changes

Repeat Step 4 (package + upload), then:

```
ssh -i "$PEM_PATH" "$REMOTE_USER@$EC2_IP" "\
cd /home/ubuntu/app/backend && \
./venv/bin/pip install -r requirements.txt -q && \
sudo systemctl restart studio-backend"
```

Quick reference: live-log commands

```
# Backend app logs, live
ssh -i "$PEM_PATH" "$REMOTE_USER@$SEC2_IP" "sudo journalctl -u studio-backend -f"

# Nginx access / error logs, live
ssh -i "$PEM_PATH" "$REMOTE_USER@$SEC2_IP" "sudo tail -f /var/log/nginx/access.log"
ssh -i "$PEM_PATH" "$REMOTE_USER@$SEC2_IP" "sudo tail -f /var/log/nginx/error.log"

# Service status
ssh -i "$PEM_PATH" "$REMOTE_USER@$SEC2_IP" "sudo systemctl status studio-backend --no-pager"
```